

简要说明文档

1. 运行环境与脚本运行方法

1.1. 运行环境

- 操作系统：Linux（WSL 环境）
- Python 版本：3.10.18（最推荐，如果版本过高 `pip install` 可能不成功）
- 依赖库版本：见 `requirements.txt`

1.2. 依赖与环境配置

使用虚拟环境进行依赖安装，命令如下：

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt # 请注意 python 版本!
```

1.3. 脚本运行方法

```
# 运行下发测试及自编测试
python tests/run_tests.py
# 性能对比测试及绘图
python performance_test/benchmark_secret_sharing.py
# 运行 demo，成功后打开相应生成端口
python src/app.py
```

运行完成后，测试结果将输出在终端中，用于验证各模块的功能正确性与整体集成情况。

2. 性能优化策略

本项目主要围绕大密钥处理与批量验证效率进行优化，目标是在保证安全性的前提下减少加密与重构的开销。

2.1. 在 `basic_shamir.py` 中支持处理最大 32KB 密钥

本方案设计了“标准模式 + 混合加密模式”双架构，实现对不同规模秘密的全覆盖支持。

对于长度 $\leq \text{block_size}$ 的小秘密（如 16 字节 AES 密钥），通过 `_split_secret_standard` 直接执行经典 Shamir 拆分，无需额外加密步骤，保持高效计算；对于长度 $\geq 1024\text{bit}$ 的大秘密，则通过 `_split_secret_hybrid` 启用混合加密：动态生成适配当前 `block_size` 的 AES 密钥（128/192/256 位自适应），用 AES-GCM 模式加密原始大秘密，仅对短 AES 密钥执行 Shamir 拆分。该方法确保了接口一致、I/O 高效和加密认证的安全性。

2.2. 在 `feldman_vss.py` 中支持批处理份额验证

为提升多份额验证的吞吐量，本方案提出一种三阶段协同策略。第一阶段，通过对承诺 C_j 预计算并缓存其幂表 i^j ，以复用昂贵的计算资源。第二阶段，对待验证集合采用双随机种子进行聚合验证，以约 $\frac{1}{q^2}$ 的极低误判率快速剪枝，概率性地剔除绝大多数无效份额。第三阶段，对聚合后剩余的少数份额，利用幂次缓存进行确定性的零误差验证，确保最终结果的准确性。该策略通过“空间换时间”的设计，将计算成本前置为可控的缓存开销，以换取批量验证场景下的高复用与低延迟，并提供配置选项以平衡在内存受限环境下的时空开销。